Lab 6

SQLite (DB Browser for SQLite)

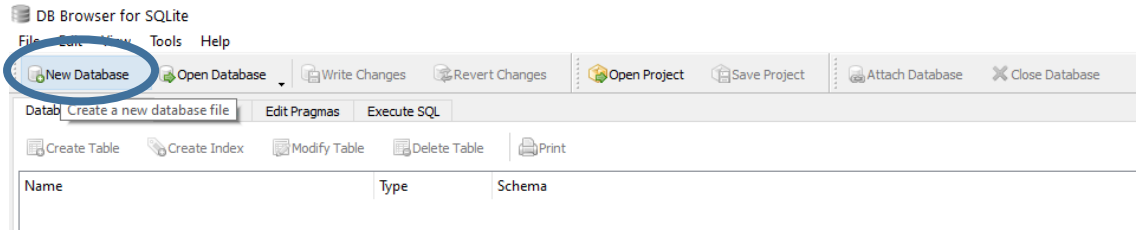
In this lab session, we are going to create a simple database using SQLite.

Our database will keep track of movies that different users have watched. The ER diagram is the following:

Figure 1 Movies Rough ER Diagram

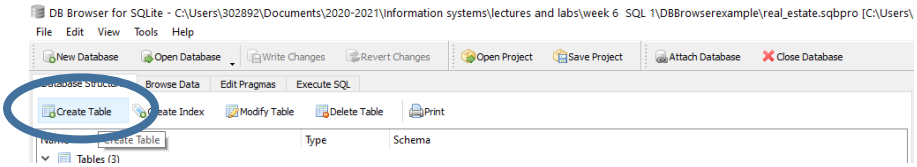
Figure 2 Movies ER Diagram

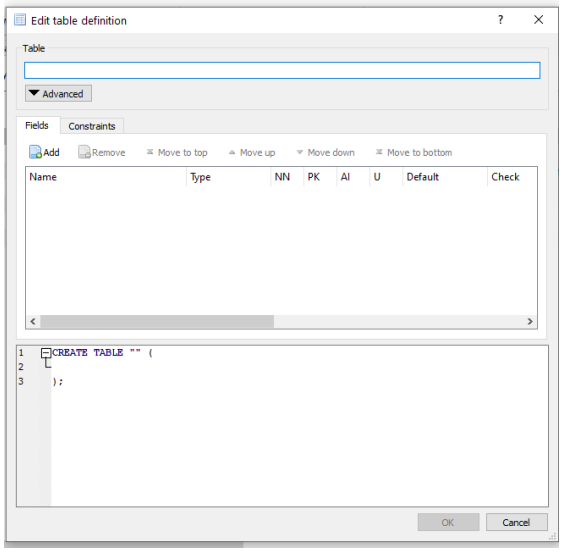
We will start opening DB Browser and creating a new database.



CREATING TABLES

USERS TABLE Create manually the first table for the *users*.





You can insert the attributes, attribute type and whether they are the primary key, allowed to be null or not, etc.. .

The correspondent SQL code will be generated in the tab below.

```
CREATE TABLE "users" (
        "user_ID"       INTEGER,
        "user_name"     TEXT,
        PRIMARY KEY("user_ID")
);
```

INSERTING DATA

Now we will enter the following information to the *users* table using the Browse Data Tab:

USERS	
1	Mary
2	Alice
3	Danny
4	Brian
5	Patricia
6	Miriam
7	Michael
8	Mareid

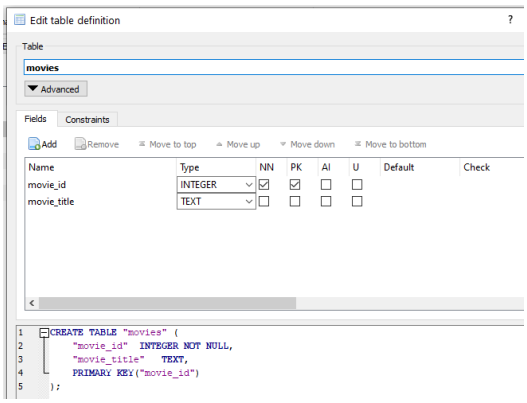
MOVIES TABLE Import table with data

For the table *movies*, we are going to import the table with its data from a csv file (download from brightspace).

Select File -> import -> Table from CSV file.

The table will be created with all its contents included.

Modify your *movies* table **selecting the primary key.**

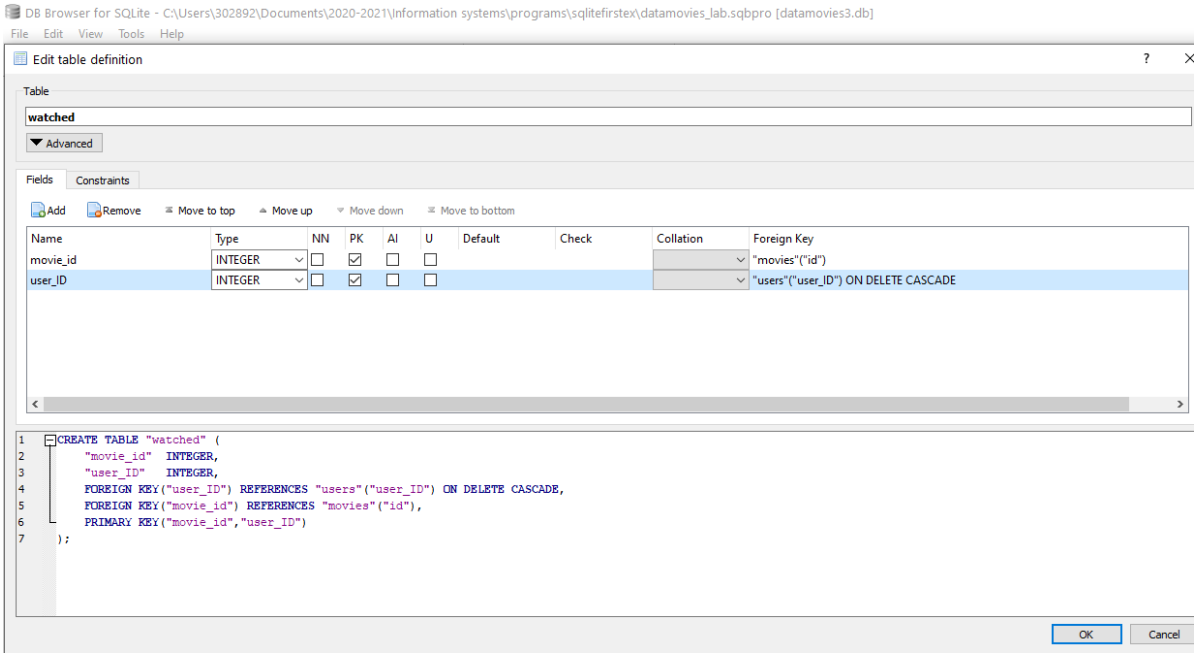


WATCHED TABLE

Once we have the *movies* and *users* tables, we will create the *watched* table. We can import the watched csv file or enter the values manually.

We will need to modify the table and include references to the *movies* and *users* tables through the foreign keys.

We will write ON DELETE CASCADE for the user_ID foreign key



SQL code created

```
CREATE TABLE "watched" (
        "movie_id"      INTEGER,
        "user_ID"       INTEGER,
        PRIMARY KEY("movie_id","user_ID"),
        FOREIGN KEY("user_ID") REFERENCES "users"("user_ID") ON DELETE CASCADE,
        FOREIGN KEY("movie_id") REFERENCES "movies"("id") ON DELETE CASCADE );
```

INSERTING DATA

Notice that by defining the foreign keys, we have connected the tables and the possible values for the cells in *watched* now come from the tables *users* and *movies*. This demonstrates the referential integrity constraint.

Try entering data in the watched table for a user that doesn't exist on the users table.

Try entering the same information in two rows. What happens?

REFERENTIAL INTEGRITY ACTIONS

When we defined the table *watched*, we included the following SQL code:

```
…
FOREIGN KEY("user_ID") REFERENCES "users"("user_ID") ON DELETE CASCADE,
FOREIGN KEY("movie_id") REFERENCES "movies"("id") ON DELETE CASCADE


…
```

Adding the referential integrity action ON DELETE CASCADE for the foreign key user_ID. This means that whenever a user is deleted, all the instances that referred to it on the *watched* table will be deleted.

Try this concept deleting a user from the database and checking the effect of this action on the *watched* table.

You can change the referential integrity to ON DELETE RESTRICT and ON DELETE NO ACTION to see the effect of the different options.

QUERIES

You can execute SQL code in the Execute SQL Tab. Perform the following queries:

1. Add extra users in bulk using SQL:
   ```
   INSERT INTO users (user_id,user_name)
   VALUES("9","Sophie"),
   ("10","Alice B."),
   ("11","Brendan"),
   ("12","David");
   ```

2. Get the titles of all the movies stored in our database:
   ```
   SELECT movie_title FROM movies
   ```

3. Get the titles of all the movies stored in our database sorted alphabetically:
   ```
   SELECT movie_title FROM movies ORDER BY movie_title
   ```

4. Get the titles of the first 5 movies stored in our database sorted alphabetically:
   ```
   SELECT movie_id, movie_title FROM movies ORDER BY movie_title LIMIT 5
   ```

5. Get a list of all users whose names start with M:
   ```
   SELECT user_name FROM users WHERE user_name LIKE "M%"
   ```

6. Get all attributes for the first 2 movies whose titles start with B:
   ```
   SELECT * FROM movies WHERE movie_title LIKE "B%" LIMIT 2
   ```

APPENDIX

NOTE on Data types for SQLite: INTEGER, TEXT, BLOB, REAL.

- **INTEGER**. The value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
- **REAL**. The value is a floating point value, stored as an 8-byte IEEE floating point number.
- **TEXT**. The value is a text string, stored using the database encoding (UTF-8, UTF-16BE or UTF-16LE).
- **BLOB**. The value is a blob of data, stored exactly as it was input.

> A Binary Large Object (BLOB) is a collection of binary data stored as a single entity in a database management system. Blobs are typically images, audio or other multimedia objects, though sometimes binary executable code is stored as a blob. Database support for blobs is not universal.

Some extra considerations

Boolean Datatype SQLite does not have a separate Boolean storage class. Instead, Boolean values are stored as integers 0 (false) and 1 (true).

Date and Time Datatype SQLite does not have a storage class set aside for storing dates and/or times. Instead, the built-in Date And Time Functions of SQLite are capable of storing dates and times as TEXT, REAL, or INTEGER values:

- **TEXT** as ISO8601 strings ("YYYY-MM-DD HH:MM:SS.SSS").
- **REAL** as Julian day numbers, the number of days since noon in Greenwich on November 24, 4714 B.C. according to the proleptic Gregorian calendar.
- **INTEGER** as Unix Time, the number of seconds since 1970-01-01 00:00:00 UTC.

For more information: https://sqlite.org/datatype3.html